



Rendli

DESIGN DOCUMENT

Business manager web application

Leads, offers, contracts, invoices and projects in one place — with an embeddable order form, online offer acceptance and contract e-signing.

Version

Date

Tech

Live

1.0

September 2026

HTML · CSS · JS · Firebase · EmailJS

maszlaig.github.io/Rendli

CONTENTS

Table of contents

01 Overview	10 Storage & security
02 Goals & audience	11 E-mail system (EmailJS)
03 Tabs & features	12 Invoicing & PDF views
04 Status workflow	13 Dashboard & charts
05 Offer	14 Bilingual & appearance
06 Contract & e-signature	15 Design system
07 Embeddable order form	16 File structure
08 Architecture & tech	17 Setup
09 Data model	18 Future work

Overview

Rendli is an account-based business-manager web app for freelancers and small businesses. It handles incoming leads, orders, projects, clients, invoices and contracts, plus income and expenses — all in one place. It also generates an **embeddable order form** whose submissions appear automatically, with no manual entry.

Data is stored per account in the cloud (**Firebase**), so the same login shows the same data on every device. The app is built with **no build step and no framework**: plain HTML, CSS and JavaScript. The interface is fully **bilingual (Hungarian / English)** and uses the shared "Studio" design with light / dark / auto appearance.

What makes it different

- **A closed loop with the client:** the offer is accepted online and the contract signed online — Rendli's status updates *automatically*, with no manual marking.
- **Embeddable form:** submissions appear live in the account without any backend of your own.
- **Card-free operation:** static hosting + Firebase free tier + EmailJS — no monthly server bill.

Rendli's philosophy: **no server of your own**. The site is served from static hosting (GitHub Pages), data lives in Firebase (Auth + Firestore), mail goes out via EmailJS — all maintenance-free services. That means low cost and simple operation.

Goals & audience

Primary user

The business owner, who signs in and manages their own business data: freelancer, sole trader, small business.

Secondary "user"

The client, who orders via the embedded form, receives an offer by e-mail, accepts it and signs a contract — with no registration.

The problem it solves

Incoming enquiries, offers, contracts and invoices typically live scattered across e-mail, paper and separate spreadsheets. It is hard to see who is where and what the next step is. Rendli ties this into a **single, status-based workflow** from lead to finished project and invoicing.

Success criteria

- Every step from lead to project happens **in one place, trackable**.
- **Manual data entry is minimal**: form submissions and client responses flow in on their own.
- Outputs (offer, contract, invoice) are **professional and branded**.
- Operation is **cheap and simple** (no backend, no monthly fee).

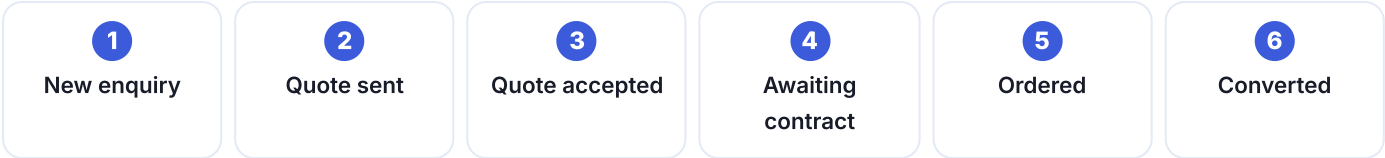
Tabs & features

The app is organised into tabs; each covers one business area.

TAB	FUNCTION
Overview	This year's revenue, ongoing and total projects, upcoming deadlines, incoming leads, this year's costs and result; monthly income/expense chart, project-status donut, monthly cash-flow table.
Orders	Incoming leads and recorded orders with a status workflow; offer and contract actions; quick price and deadline editing.
Projects	Ongoing and closed work; created automatically from an order, with its own statuses.
Clients	The client base, built automatically from leads and orders.
Invoices	Invoice issuing and a printable PDF view (a clean print window with no browser header/footer).
Contracts	Assignment-contract templates (web, content, photo, construction), viewable and saveable as PDF; online signing.
Income / Expenses	Recording and summarising financial items, with categories.
Form	Configuring the embeddable order form (services, fields, prices, language) and copying the generated code.
Account	Profile, business data, appearance (light/dark/auto), language (HU/EN), export/import, reset.

Status workflow

A job is tracked through the lead status, with well-defined transitions.



STATUS	MEANING · RULE
New enquiry	Received, not yet processed.
Quote sent	The itemised offer has been e-mailed.
Quote accepted	The client accepted (online or marked manually). This gates the contract.
Awaiting contract	The contract is out; we await the signature.
Ordered	Signed; the lead is automatically added to Projects.
Converted to project	After manual conversion; the order lives on in the Projects tab.

A **contract can only be created** after the offer has been sent *and* accepted. This gate is enforced from both the status dropdown and the button, preventing skipped steps.

Offer

Compose an itemised quote, send it, and let the client accept in one click.

Composition

Line items (name, quantity, unit price), automatic **net / VAT / gross** totals per VAT status, an optional message and a validity date. The e-mail goes out in the business's branding, with a summary table.

PDF download

A **"Download as PDF"** button in the e-mail opens the standalone `ajanlat.html` page, where the client can view and save the offer as PDF. The offer data travels in the link's `#` fragment (base64) — **no backend or storage needed**.

Online acceptance

On the same page an **"Accept"** button writes the acceptance into the Firestore inbox in one click; the account pulls it in live and the status becomes "Quote accepted" automatically.

Security: acceptance happens **only on an explicit click**, never on page load — so e-mail link-preview bots cannot accidentally accept it.

Contract & e-signature

Compose an assignment contract from a template, send it, and let the client sign online by drawing.

Templates & clauses

Several contract types (website, content, photo, construction) with a full clause set: fee & payment, delivery, licence, confidentiality, GDPR, termination, force majeure, closing provisions. Bracketed [] parts can be refined in the print view.

Signing page

A **"Sign & accept"** button in the contract e-mail opens the standalone `szersizodes.html` page. The contract loads from Firestore (via a random, unguessable token); the client **draws their signature** (finger or mouse) and enters their name. The signature is placed above the Client's signature line in the document.

Write-back & automation

The signature (image, name, timestamp) returns to the account through the Firestore inbox; the status becomes **"Ordered"** and the lead is **automatically added to Projects**. The signed copy is stored in the `contracts` collection and can be viewed / saved as PDF anytime.

Legal note: this is a **documented electronic signature** (drawn mark + name + timestamp + the exact contract text stored). It is sufficient and provable for most freelance/SMB engagements, but it is **not a qualified (eIDAS) electronic signature**.

Embeddable order form

On the Account → Form tab you configure an order form, for which Rendli generates an **embeddable snippet**. It can be placed on any external website (e.g. your own portfolio) and follows the host page's language.

ELEMENT	BEHAVIOUR
Configuration	Services (names, prices HUF/EUR), fields (phone, message, deadline, budget), business/individual toggle.
Submission	The order goes into the account inbox (<code>inbox/<uid>/items</code>), with a size limit.
Live pull	The app watches the inbox with <code>onSnapshot</code> ; a new submission is pulled into leads and removed from the queue.
Confirmation	On submit, two e-mails: to the client ("Thank you") and to the owner ("New order").
Styling	The confirmation dialog inherits the host page's CSS variables, matching the embedding site.

Architecture & tech

Client

Vanilla HTML/CSS/JS, no build step. Custom canvas charts, no external chart library.
Dictionary-based HU/EN i18n, light/dark/auto theme.

Cloud

Firebase Authentication (email/password) + Cloud Firestore. EmailJS for outbound mail.
Static hosting (GitHub Pages) for the site.

Modules

FILE	RESPONSIBILITY
firebase-store.js	Firebase Auth + Firestore data layer (LocalStore wrapper); loads/saves vaults/<uid> .
script.js	Auth flow, shared state, dashboard, invoicing (+ PDF), form generator, dialogs, i18n.
leads.js	Inbox watcher, leads/orders module, offer sending + acceptance, auto project creation.
contract.js	Contract builder, sending, signing flow and PDF view.
theme.js , fit-text.js	Theme handling; auto-fit for large numeric displays.

Data model

Each user's full state lives in a single Firestore document (`vaults/<uid>`) as one large state object.

```
// vaults/<uid> - main parts of the state
{
  profile, sellerInfo,           // business data (name, tax id, bank...)
  leads: { <id>: { status, offer, contract, ... } },
  orders: [ ... ],              // projects
  invoices: [ ... ],
  bizIncome: [ ... ], bizExpense: [ ... ],
  inboxKey, formConfig,         // embedded form
  fxEurHuf, bizTaxRate, ...
}
```

Key fields of a lead

status	new · offer · accepted · contract · ordered · converted
offer	items, totals, validity, sentAt, accepted, acceptedAt
contract	contractId, sentAt, signed, signerName, signedAt
base	name, email, phone, type, price, deadline, date

Storage & security

Private data belongs only to the owner; public channels are tightly scoped with least privilege.

COLLECTION	ACCESS
<code>vaults/<uid></code>	Read/write only for the signed-in owner (their own uid).
<code>form_configs/<uid></code>	Anyone can read (for the widget); only the owner writes.
<code>inbox/<uid>/items</code>	Anyone can create (size-limited); only the owner reads/deletes.
<code>contracts/<uid>/docs</code>	Public read via the random token; only the owner writes.

Defence principles

- **Least privilege:** public channels allow only create/read, never modification of the owner's data.
- **Unguessable token:** contracts are reachable only via the random link.
- **Explicit gesture:** acceptance/signing happens on click only — link-preview bots cannot trigger it.
- **Public keys:** the Firebase and EmailJS keys are browser-public identifiers; private/secret keys never reach the client.

The rules (`firestore.rules`) live in the `firebase/` folder and deploy via the Firebase CLI: `cd firebase && firebase deploy --only firestore:rules` .

E-mail system (EmailJS)

Outbound mail is sent from the browser, with no server of your own.

MAIL	ACCOUNT	TRIGGER
Client confirmation	A	Form submit
Owner notification	A	Form submit
Offer (PDF + accept)	B	Send offer
Contract (online sign)	B	Send contract

Two accounts

The free EmailJS plan allows 2 templates per account, so there are **two accounts**: "A" serves the embedded form's confirmations, "B" the app-sent offer and contract. Each template can only be sent with its own account's keys.

Template rule

Templates use **no `{{#if}}` blocks** (EmailJS's base engine doesn't support them — it returns a "corrupted variables" error). Conditional buttons are built in code as ready HTML and injected via raw `{{{...}}}` variables.

EmailJS setting: keep "Allow EmailJS API for non-browser applications" off, otherwise browser (public-key) sending returns 403.

Invoicing & PDF views

Invoices, contracts and offers render as clean, printable PDFs — with no external library.

- **Invoice:** issued on the Invoices tab; the "PDF" button opens a clean print window (no browser header/footer) that the user saves as PDF.
- **Contract:** viewable and saveable as PDF; the signed copy includes the drawn mark.
- **Offer:** the client saves it as PDF on the standalone `ajanlat.html` page.

Print views use `@media print` and `@page` rules to render precisely on A4.

Dashboard & charts

The Overview tab summarises the state of the business with custom (canvas) charts.

Income / expense

Monthly bar chart of this year's cash movement.

Project status

Donut of project distribution.

Cash flow

Month-by-month cash-flow table.

Large numeric displays are auto-sized by `fit-text.js` to always stay readable.

Bilingual & appearance

Bilingual (HU/EN)

Dictionary-based i18n; the whole interface is available in Hungarian and English. The chosen language is remembered in the browser.

Theme

Light / dark / auto appearance (`data-theme`); auto follows the system setting. The choice lives in `localStorage` .

The e-mail templates and standalone pages (offer, contract) carry the same branding, giving the client a consistent experience throughout.

Brand & design system

The shared "Studio" design: a clean, light surface with an indigo accent and the Inter typeface.

Colour palette

					
Indigo (accent) #3b5bdb	Ink (text) #171c28	Muted text #5d6b85	Background #eef1f8	Line / border #e4eaf5	Green (success) #1e7a34

Status colours

				
New — red	Offer — cyan	Accepted — yellow	Contract — purple	Ordered — green

Typography

Heading	Inter · 800 · tight tracking
Emphasis	Inter · 700
Body	Inter · 400 · 1.55 line-height
Mono / code	JetBrains Mono

Components

- **Card:** white, 1px light border, 12px radius.
- **Button:** solid indigo (primary), white+indigo border (secondary), solid green (accept/sign).
- **Status badges:** colour-coded pills. **Tables:** hairline separators, invoice-style totals.
- **Callouts:** left accent bar (indigo / green / amber).

File structure

```
04_Rendli/  
├─ index.html      - the full UI  
├─ ajanlat.html    - public offer page (PDF + accept)  
├─ szerzodes.html  - public contract-signing page  
├─ README.md  
├─ css/            - "Studio" design  
├─ js/             - app logic (script, leads, contract, ...)  
├─ e-mail sablonok/ - EmailJS templates  
├─ firebase/       - Firebase config + rules (CLI)  
└─ dokumentumok/   - notes, this design document
```

Setup

1. **Firestore:** create a project, enable Authentication (email/password), fill the config in `index.html`, deploy the Firestore rules from the `firebase/` folder.
2. **EmailJS:** set up the two accounts and four templates, paste the keys/template IDs into the code.
3. **Publish:** upload the files to static hosting (GitHub Pages); keep `ajanlat.html` and `szerzodes.html` at the root.
4. **First use:** register an account, fill in business data, configure the form, copy the generated code onto your site.

Checklist

- Firestore rules deployed (`cd firebase && firebase deploy --only firestore:rules`).
- All four EmailJS templates saved, with the Settings fields.
- Test order, offer and contract run end-to-end.

Detailed steps: `dokumentumok/BEALLITAS-Firebase.txt` and `dokumentumok/BEALLITAS-EmailJS.txt` .

Future work

Possible directions, keeping the current card-free philosophy.

- Automatic payment reminders and invoice-status tracking.
- A calendar view for deadlines and booked capacity.
- A client portal: the client sees the project status on their own secure link.
- More languages and currencies for international clients.
- Qualified (eIDAS) e-signature integration with an external provider where legally required.

Thanks to Rendli's modular, framework-free structure, these can be added gradually without disrupting the existing operation.